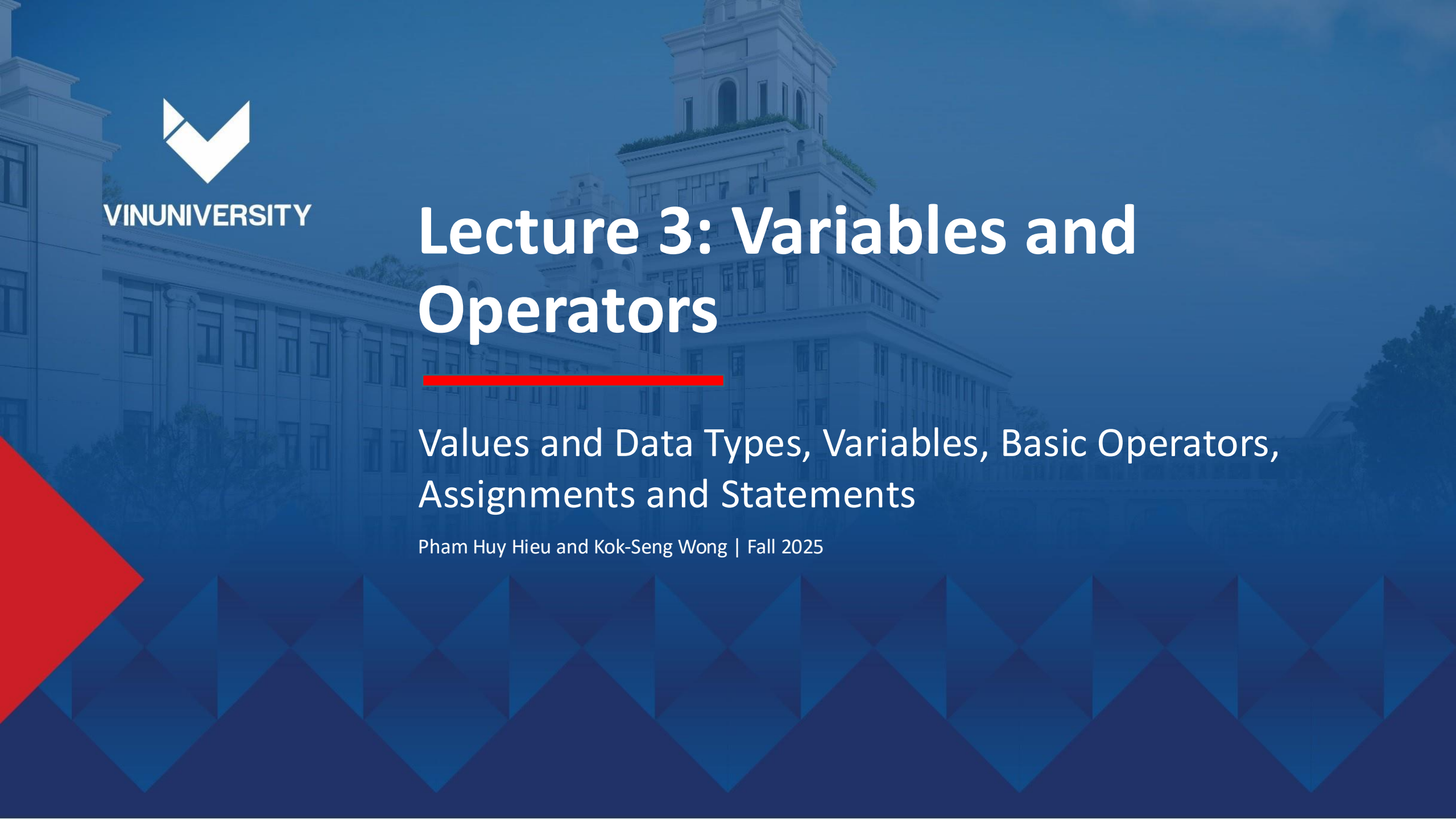


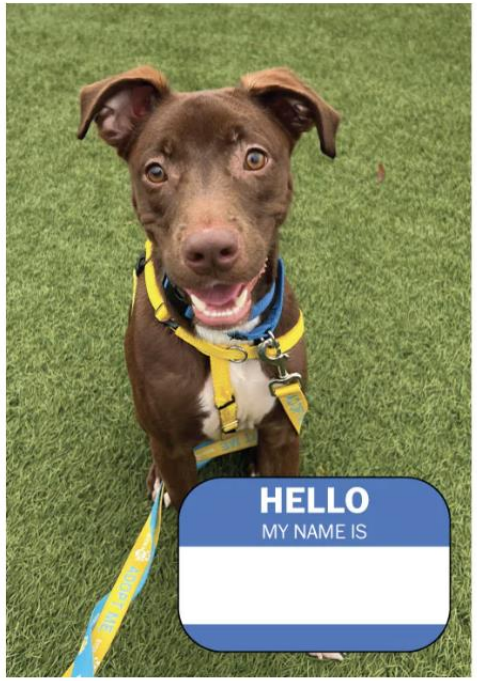
VINUNIVERSITY

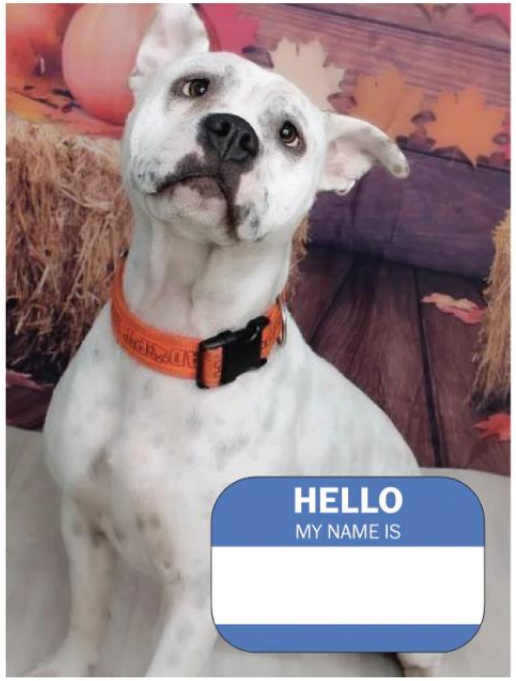
Lecture 3: Variables and Operators

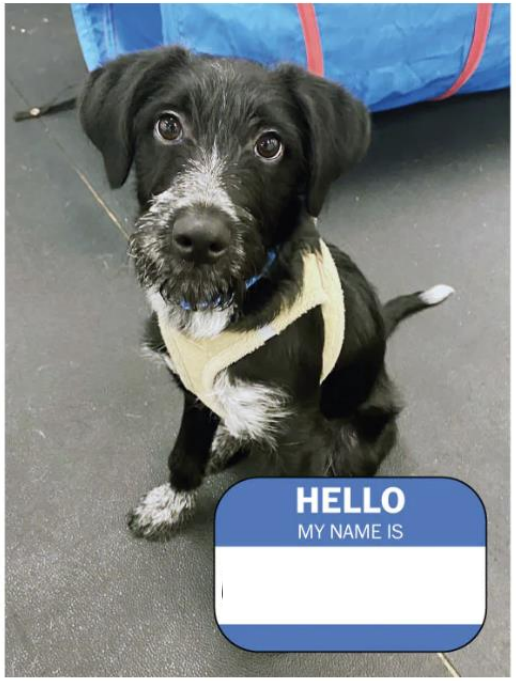
Values and Data Types, Variables, Basic Operators, Assignments and Statements

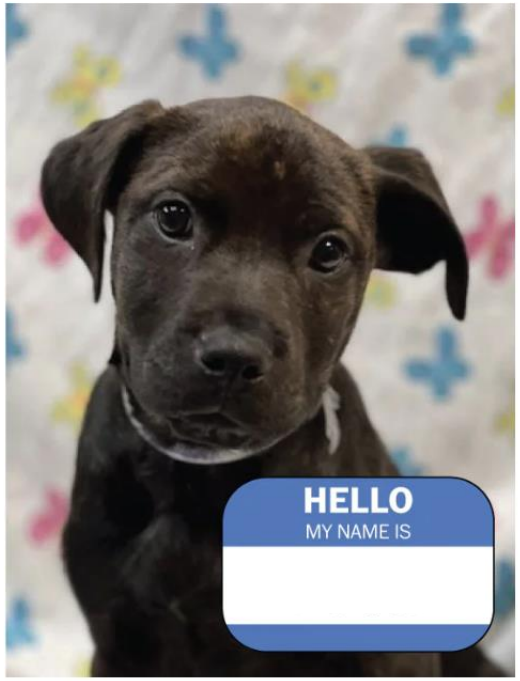
Pham Huy Hieu and Kok-Seng Wong | Fall 2025

MOTIVATIONAL EXAMPLE 1



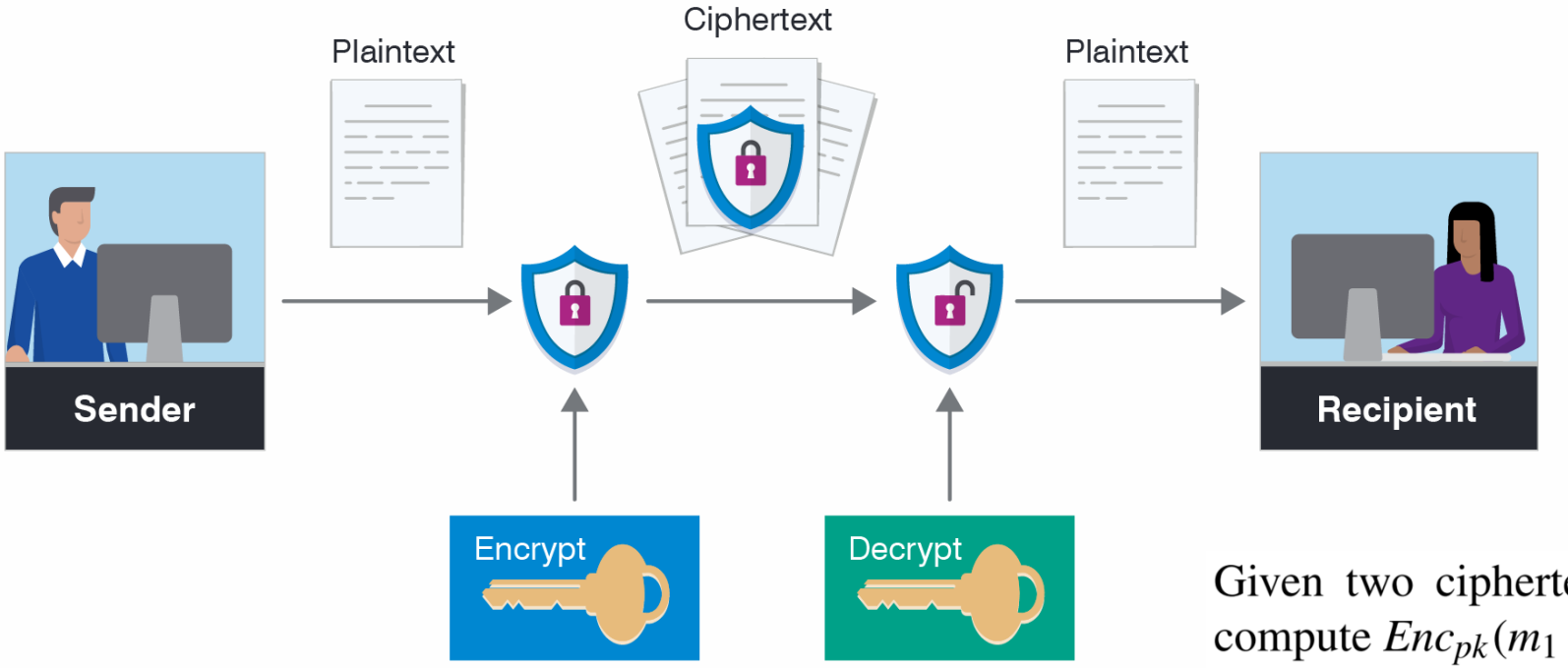






https://www.washingtonpost.com/business/interactive/2022/people-names-for-dogs/

MOTIVATIONAL EXAMPLE 2



Different keys are used to encrypt and decrypt messages

[source]

$$\begin{aligned}\mathcal{P}(\mathcal{E}) &= \mathcal{P}\left(\bigcup_{i=1}^{s}\left[\{X_1 = i\}\bigcap\{X_2 = i\}\right]\right)\\ &= \sum_{i=1}^{s}\mathcal{P}\left(\{X_1 = i\}\bigcap\{X_2 = i\}\right)\\ &= \sum_{i=1}^{s}\mathcal{P}(X_1 = i)\mathcal{P}(X_2 = i|X_1 = i)\\ &= \sum_{i=1}^{s}\left(\frac{k}{M}\right)^2 = \frac{k}{M}.\end{aligned}$$

Given two ciphertexts $Enc_{pk}(m_1)$ and $Enc_{pk}(m_2)$, we can compute $Enc_{pk}(m_1 + m_2)$ as follows:

$$\begin{aligned}&Enc_{pk}(m_1) +_h Enc_{pk}(m_2) \bmod N^2\\ &= ((1+N)^{m_1} r_1^N) +_h (1+N)^{m_2} r_2^N) \bmod N^2\\ &= ((1+N)^{m_1+m_2}(r_1+r_2)^N \bmod N^2\\ &= Enc_{pk}(m_1 + m_2) \bmod N^2\end{aligned}$$

LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- understand python's storage model and operator precedence.
- distinguish between expressions values, and printed representations.
- input function.
- use different operations on string.

BASIC DATA TYPES (REVIEW)

- **Integer** (int) and **float** (float) - an integer or a real number in mathematics

```
1000000, 0.143, 54, 7644
```

- **String** (str) - sequences of character data

```
"Who are you?", "100", "0.143", ""
```

- **Boolean** (bool) - truth values (True or False)

```
10 > 9?, 10 < 9?
```

```
>>> type(54)
<class 'int'>
>>> type(0.143)
<class 'float'>
>>> type("0.143")
<class 'str'>
>>> type(10>9)
<class 'bool'>
>>> type("")
<class 'str'>
```

TYPE CONVERSION (1/2)

The int function takes any value and converts it to an integer:

- String to integer

```
>>> a = "123"
>>> type(a)
<class 'str'>
>>> int(a)
123
>>> type(int(a))
<class 'int'>
```

```
>>> int("Hi!")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int()
with base 10:'Hi!'
```

- Floating-point values to integer

```
>>> int(3.142)
3
>>> int(-3.2)
-3
>>> int(0.9999)
0
```

It truncates the fractional part!

TYPE CONVERSION (2/2)

- The float function converts integers and strings to floating-point numbers:

```
>>> type(123)
<class 'int'>
>>> type(float(123))
<class 'float'>

>>> type("123")
<class 'str'>
>>> type(float("123"))
<class 'float'>
```

- The str function converts to type string:

```
>>> type(123)
<class 'int'>
>>> type(str(123))
<class 'str'>

>>> type(3.142)
<class 'float'>
>>> type(str(3.142))
<class 'str'>
```

HOW DOES PYTHON COMPUTE THIS?

```
>>> 2.1 / 4
```

(A) Convert 2.1 into the integer 2, then calculate 2/4 = 0.5

(B) Convert 2.1 into the integer 2, then calculate 2//4 = 0

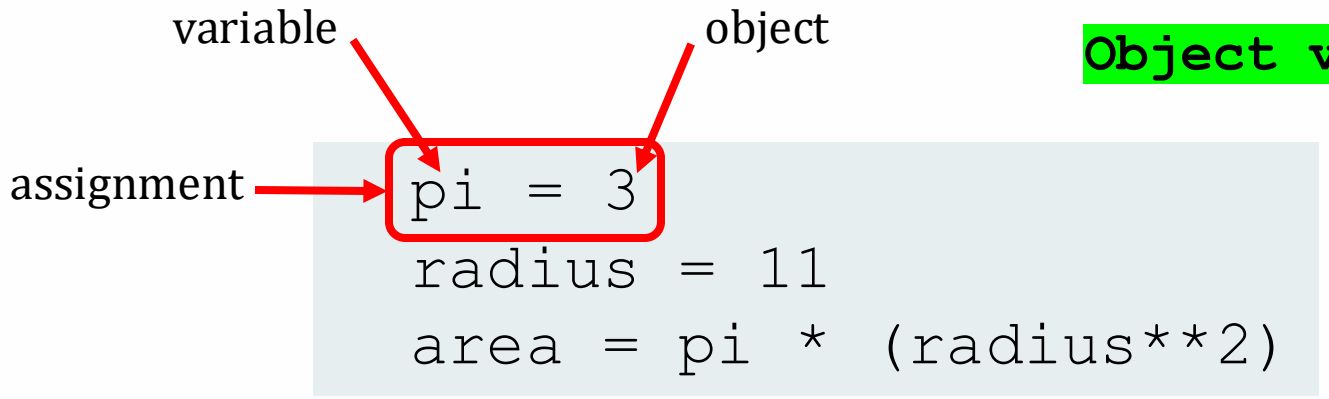
(C) Convert 4 into the float 4.0, then calculate 2.1/4.0 = 0.525

(D) Produce a Type Error

```
>>> 4 / 2
```

VARIABLES & ASSIGNMENT

- In Python, **a variable is just a name**, nothing more!
- Variables need not be declared (or defined) in advance in Python.
- To create a variable, simply assign it a value.
- **Assignment:** variables provide a way to associate names with objects.



Object vs Value?

Python allows multiple assignment

```
pi, radius = 3, 11
```

PYTHON OBJECT

- An **object** is an instance of a data structure that holds both data (state) and methods (behavior).
- Everything in Python is an object, including numbers, strings, functions, classes, and even modules.
- Objects have an **identity**, **type**, and **value**.
  - **Identity**: This is a unique identifier for the object (its memory address). You can check an object's identity using the `id()` function.
  - **Type**: The type of the object defines what kind of object it is (e.g., integer, string, list). You can check an object's type using the `type()` function.
  - **Value**: The data stored within the object. Depending on the type, this could be a number, a string, or a more complex structure like a list.

```
x = 5
print(id(x))   # identity
print(type(x)) # type
print(x)       # value
```

ChatGPT

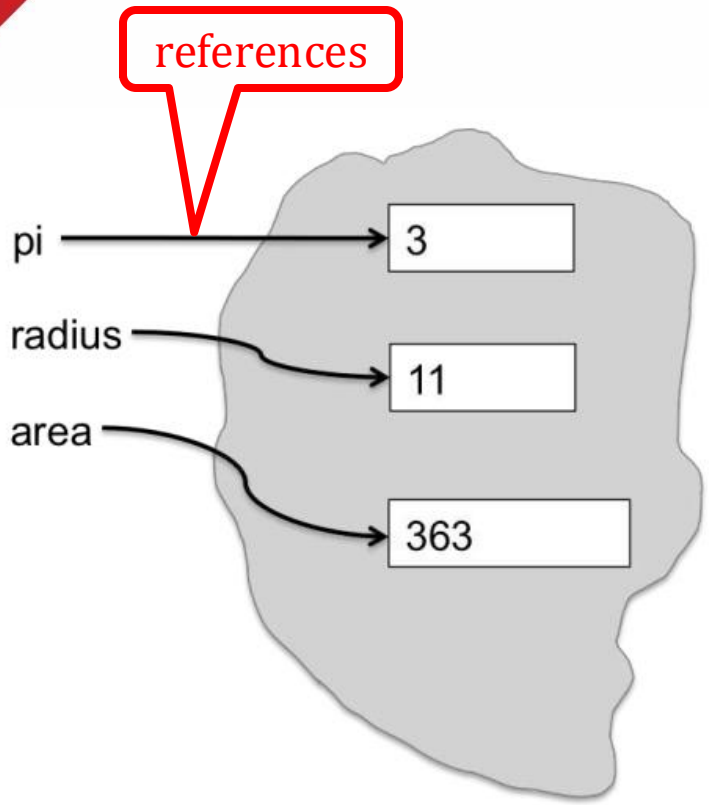
PYTHON VALUE

- The **value** is the actual data that an object holds. For immutable objects (like integers, strings, or tuples), the value cannot be changed after the object is created.
- For mutable objects (like lists or dictionaries), the value can be altered without creating a new object.
- When you work with Python variables, you are typically working with references to objects, not the values directly.

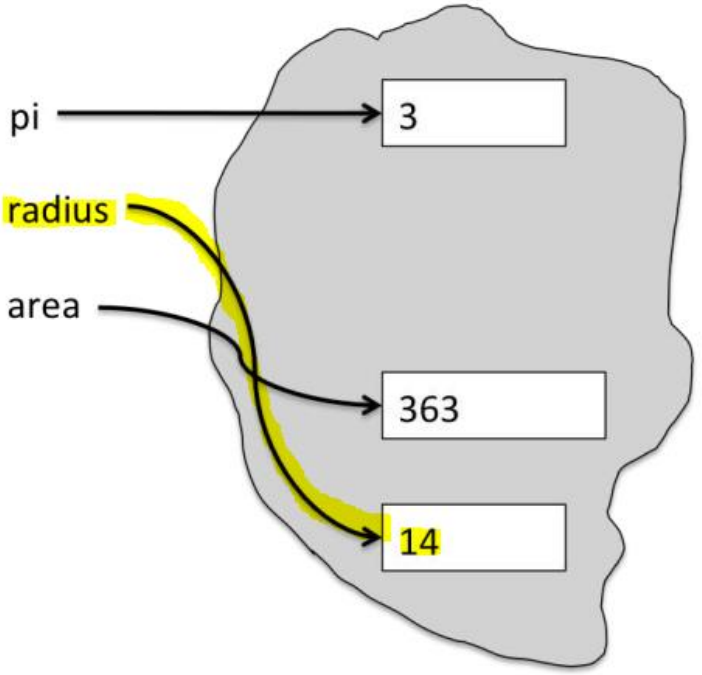
```
a = 10  # `a` references an integer object holding the value 10
b = a   # `b` now also references the same object as `a`
a = 20  # `a` now points to a new object with the value 20
print(b) # `b` still references the original object with value 10
```

ChatGPT

BINDING OF VARIABLES TO OBJECTS




```
pi = 3
radius = 11
area = pi * (radius ** 2)
radius = 14
```



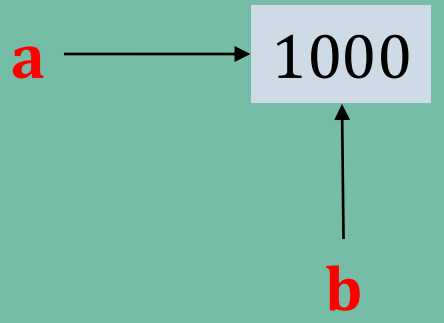

```
radius = 11
print(id(radius))
radius = 14
print(id(radius))
```

```
140185182464624
140185182464720
```

CLASS PRACTICE

```
>>> a = 1000
>>> b = a
```

Python has created how many objects and how many references?



CLASS PRACTICE

```
>>> x, y = 2, 3
>>> x, y = y, x
>>> print ('x =' ,x)
>>> print (‘y =' ,y)
```

What are the outputs?

CHOOSING APPROPRIATE VARIABLE NAMES



NAMING RULES

- Can contain uppercase and lowercase letters, digits (but they cannot start with a digit), and the special character _

```
firstName, Group5, _myname
```

- Case-sensitive:

```
VinUni, vinUni, Vinuni
```

- Cannot use **reserved words** (also called **keywords**) that have built-in meanings.



```
>>> help()

Welcome to Python 3.8's help utility!

If this is your first time using Python, you should definitely check out
the tutorial on the Internet at https://docs.python.org/3.8/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules.  To quit this help utility and
return to the interpreter, just type "quit".

To get a list of available modules, keywords, symbols, or topics, type
"modules", "keywords", "symbols", or "topics".  Each module also comes
with a one-line summary of what it does; to list the modules whose name
or summary contain a given string such as "spam", type "modules spam".
```



```
help> keywords

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               class               from                or
None                continue            global              pass
True                def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
break               for                 not
```

CLASS PRACTICE

Which of the following are valid Python variable names?

```
>>> 1VinUni = "My Uni"

>>> more$ = 1000

>>> class = "CS 101"

>>> ver1.3 = "Aug21"
```

OPERATORS (REVIEW)

- **Operators** are special symbols that represent computations like addition (+), subtraction (–), multiplication (*) and division (/).
- The values the operator uses are called **operands**.

```
20+32, hour-1, minute/60, (5+9)*(15-7), 5**2
```

exponential

ARITHMETIC OPERATORS (REVIEW)

Operator	Description	Example	Shortcut
+ Addition	Adds values on either side of the operator.	`a + b = 30`	`x += y`
- Subtraction	Subtracts right hand operand from left hand operand.	`a - b = -10`	`x -= y`
* Multiplication	Multiplies values on either side of the operator	`a * b = 200`	`x *= y`
/ Division	Divides left hand operand by right hand operand	`b / a = 2.0`	`x /= y`
% Modulus	Divides left hand operand by right hand operand and returns remainder	`b % a = 1`	`x %= y`
** Exponent	Performs exponential (power) calculation on operators	`a**b 10 to the power 20`	`x **= y`
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity)	`9//2 = 4, 9.0//2.0 = 4.0, -11//3 = -4, -11.0//3 = -4.0`	`x //= y`

source

COMPARISON OPERATORS (REVIEW)

Operator	Description	True or False? (assume a=10 and b=20)
==	If the values of two operands are equal, then the condition becomes true.	(a == b)
!=	If values of two operands are not equal, then condition becomes true.	(a != b)
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	(a > b)
<	If the value of left operand is less than the value of right operand, then condition becomes true.	(a < b)
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	(a >= b)
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	(a <= b)

source

BITWISE OPERATORS (REVIEW)

Bitwise operators act on operands as if they were strings of binary digits.

Operator	Meaning	Example
`&`	Bitwise AND	`x & y`
`|`	Bitwise OR	`x | y`
`~`	Bitwise NOT	`~x`
`^`	Bitwise XOR	`x ^ y`
`>>`	Bitwise right shift	`x >> 2`
`<<`	Bitwise left shift	`x << 2`

ASSIGNMENT OPERATORS (REVIEW)

Operator	Example	Equivalent to
=	x = 3	x = 3
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

ORDER OF OPERATIONS

When more than one operator appears in an expression, the order of evaluation depends on the **rules of precedence**.

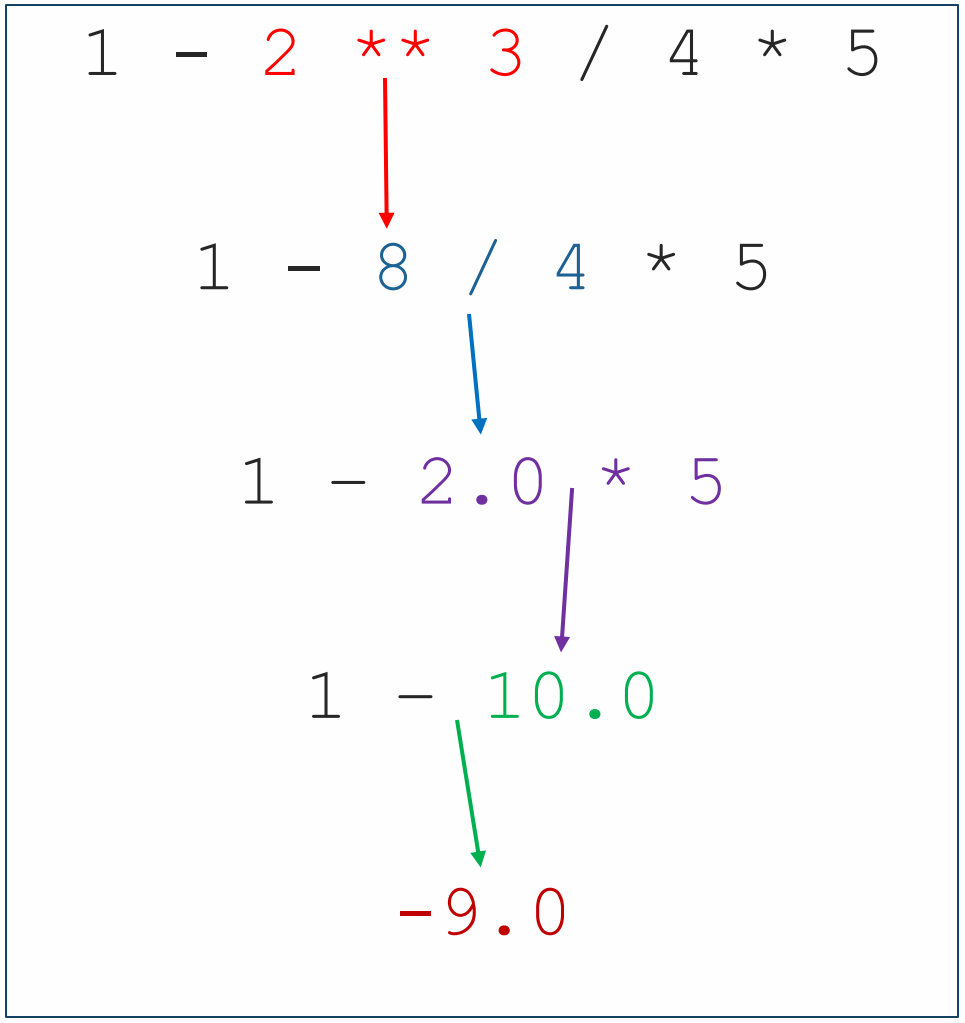
Highest

Lowest

Operators	Meaning
( )	Parentheses
**	Exponential
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity, Membership operators
Not	Logical NOT
and	Logical AND
or	Logical OR

EXAMPLE 1

```
>>> x = 1 - 2 ** 3 / 4 * 5
>>> print(x)
???
```



CLASS PRACTICE

Write a Python codes to print the output of the following equation if given $x = 8$

$$y = 4x^3 + 3x^2 - x + 6$$

```
>>> x = 8
>>>
```

NUMBER BASES

Decimal (10)	Binary (2)	Octal (8)	Hexadecimal (16)
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

- 0b or 0B for Binary (base 2)
- 0o or 0O for Octal (base 8)
- 0x or 0X for Hex (base 16)

```
>>> 0b1000
8
>>> 0o10
8
>>> 0x8
8
```

BINARY, OCTAL AND HEXADECIMAL (1/2)

- Representation of numbers (integer) in binary, octal, and hexadecimal:

```
0b1000111 = 71    0o1010101 = 266305    0x1000100 = 16777472
```

- To convert a number to binary, octal or hexadecimal string:

Built-in functions

- `bin()`
- `oct()`
- `hex()`

```
>>> bin(71)
'0b1000111'
>>> oct(266305)
'0o1010101'
>>> hex(16777472)
'0x1000100'
```

BINARY, OCTAL AND HEXADECIMAL (2/2)

- To convert binary, octal or hexadecimal string to integer:

```
>>> bin_numb = 0b1000111
>>> oct_numb = 0o1010101
>>> hex_numb = 0x1000100
>>> print(bin_numb)
71
>>> print(oct_numb)
266305
>>> print(hex_numb)
16777472
```

CLASS PRACTICE

```
>>> x, y = 0b1010,0b100
>>> x, y
>>> x & y
>>> x | y
>>> ~x
>>> x ^ y
>>> x >> 2
>>> x << 2
```

ASCII VALUE

- ASCII: American Standard Code for Information Interchange.
- Each character (and symbol) has a given numeric value that computer used to store and manipulate.
- ASCII value: 0 and 127
- Example:
  - ‘A’ is 65
  - ‘a’ is 97
  - ‘Z’ is 90
  - ‘z’ is 122

ASCII TABLE

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	|
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

© w3resource.com

FIND ASCII IN PYTHON

- `ord()`: converts a character into Unicode code value

ordinal value

```
>>> ord('A')
65
>>> ord('a')
97
>>> ord('Z')
90
>>> ord('z')
122
```

- `chr()`: returns the character that represents the Unicode code value

```
>>> chr(100)
'd'
>>> chr(107)
'k'
>>> chr(87)
'W'
```

CLASS PRACTICE

```
>>> chr(ord('A')+10)

>>> chr(ord('d')+3)

>>> chr(ord('k'))
```

OPERATIONS ON STRINGS (REVIEW)

- Concatenation of string: + operator

```
>>> first_name = "Bond"
>>> last_name = "James"
>>> last_name + first_name
'JamesBond'
```

- Concatenation of string: * operator

```
>>> last_name * 3
'JamesJamesJames'
>>> first_name * 4
'BondBondBondBond'
```

3*3=3+3+3

last_name+last_name+last_name

- Concatenation of string: print( )

```
>>> print(last_name, first_name)
James Bond
```

INPUT FUNCTION

- syntax: `input("")`
- To accept user's input (i.e., user types in something and hits enter)
- Whatever users enter as input, the `input()` converts it into a **`string`**

```
>>> last_name = input("Enter your last name: ")
Enter your last name: Baba
>>> first_name = input("Enter your first name: ")
Enter your first name: Ali
>>> print(first_name, last_name)
Ali Baba
```

HOW TO TAKE INTEGER INPUT?

```
>>> stud_ID = input("Enter your ID: ")
Enter your ID: 20024512
>>> stud_ID + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int")
to str
>>> type(stud_ID)
<class 'str')
```

```
>>> score = int(input("Enter your score: "))
Enter your score: 88
>>> type(score)
<class 'int')
```

CLASS PRACTICE

```
>>> age = input("What is your age? ")
>>> print(age)
>>> print(2 * age)
```

What is the outputs if age=10?

CLASS PRACTICE

Write a python program to calculate the value of F based on the user's inputs:

$$F = G\frac{m_1 m_2}{r^2}$$